ReSumo Platform Technical Architecture

End-to-End System Execution Flow: Ingestion, Full-SFT Parser, Embeddings, Database & Semantic RAG

Project Title: AI-Driven Recruitment & Onboarding Automation with Human-in-the-Loop Approval
Course: CSD 415 — Project Evaluation | **Department:** Computer Science & Engineering
Supervisor: Dr. Sumod Sundar (Associate Dean, Department of Computer Science)
Authors: Niya Susan Rajeev (PRC23CS074), Murali Krishnan R (PRC23CS070), Yadhu Krishnan R (PRC23CS088), Jan Cherian Thomas (PRC23CS052)
Evaluation Date: October 28, 2026

1. Chronological Pipeline Execution Overview

Stage	Pipeline Stage	Core Technology	Execution Details & State Artifacts
01	Upload Gateway	FastAPI / UUIDv4	File validation (PDF/DOCX, <5MB), unique disk write.
02	Text & OCR Engine	pypdf / Tesseract v5	Dual-mode parsing (<10 words triggers OCR fallback).
03	Full SFT LLM Parser	Qwen2.5 (Full SFT)	Strict extraction into standardized 15-key JSON schema.
04	Transactional DB	PostgreSQL / SQLAlchemy	Candidate row saved with 'parsed' status and JSONB profile.
05	Vector Embedding	BAAI/bge-large-en-v1.5	Encodes structured candidate profile into 1,024-dim vector.
06	Vector Indexing	ChromaDB (HNSW)	Inserts normalized vector in Cosine Space for ANN lookup.
07	Job Embedding	FastAPI / bge-large	Embeds Job Description into 1,024-dim query vector.
08	Semantic Matching	Cosine Metric Space	Calculates contextual match % (Score = 1 - Cosine Distance).
09	Explainable RAG	Grounded LLM Generator	Generates human-readable fit rationale (strengths vs gaps).
10	Tiered HITL Signoff	Hierarchical State Machine	Policy escalation: Junior/Mid -> HM, Senior -> VP, Exec -> CEO.

2. Stages 1 & 2: Document Ingestion & Dual-Mode OCR Fallback

Source Files: backend/app/routers/resumes.py, backend/app/services/parser.py, backend/app/services/contact_extraction.py

- **File Reception & Security:** The endpoint POST /api/resumes/upload-resume validates MIME types (.pdf, .docx, .png, .jpg) and enforces a strict 5MB payload limit to prevent buffer exhaustion attacks.
- **Atomic Disk Storage:** Generates a collision-free UUIDv4 identifier and writes the binary stream to uploads/resumes/<uuid><ext>.
- **Dual-Mode Text Extraction:** The service extract_text() reads programmatic text via pypdf or python-docx. **OCR Trigger Condition:** If extracted text length is < 10 words (scanned image, photocopied resume, flattened graphic PDF), it automatically invokes **Tesseract v5 OCR** with PIL image binarization.
- **Contact Extraction:** Runs deterministic regular expressions in extract_contact_info() to isolate Name, Email, Phone, and Portfolio links, creating the initial Candidate record in PostgreSQL with status 'uploaded'.

3. Stage 3: Full Supervised Fine-Tuned (Full SFT) Resume Parser

Source Files: backend/app/services/extraction.py, backend/app/services/qwen_prompt.py

Why Generalist LLMs Fail for Recruitment Ingestion: Off-the-shelf foundation models (GPT-4, base Qwen) exhibit three core deficiencies in automated recruitment pipelines: (1) *Conversational verbosity* (emitting polite chatter like 'Here is your JSON:') that causes JSON decoding crashes; (2) *Hallucination* of missing past companies or skill proficiencies; and (3) *Schema key drift* (e.g. returning 'exp' or 'skills_list' instead of standardized schema keys).

Fine-Tuning Methodology & Mathematical Formulation

- **Base Foundation Model: Qwen2.5** Causal Language Model, pre-trained on diverse multilingual text and high-density technical code.
- **Training Dataset:** sandeeppanem/resume-json-extraction-5k on Hugging Face (5,000 real-world, diverse resumes paired with human-verified 15-key ground truth JSON extractions).
- **Data Sanitization:** Automated preprocessing pipeline (clean_html) strips HTML/DOM artifacts, unescapes entities, and collapses erratic line breaks into clean text.
- **Full SFT Paradigm:** Unlike parameter-efficient adapters (LoRA/QLoRA) that only train low-rank decomposition matrices ($r=16$), **Full Supervised Fine-Tuning updates 100% of all transformer parameters across all layers**. This embeds recruitment domain concepts, non-linear career trajectories, and strict JSON grammar directly into the model's core attention weights.
- **Loss Function & Training Objective:** Standard auto-regressive Cross-Entropy Loss computed exclusively over the target JSON tokens (prompt tokens masked out):

Full SFT Cross-Entropy Loss:

$$L_{SFT}(\theta) = - \sum_{t=1}^{T} \log P(y_t \mid y_{<t}, x_{resume}; \theta)$$

Where x_{resume} is the input resume text, y_t are the ground-truth JSON tokens, and θ represents all model weights.

- **Local Quantized Serving:** Full fine-tuned model weights were exported and quantized to 8-bit GGUF format (resume-parser-q8_0.gguf) and served locally via llama-server / LM Studio on Port 8001/1234. **Privacy Advantage:** Zero candidate PII leaves the local host, guaranteeing complete GDPR compliance and data privacy.
- **Non-Blocking Execution:** Invoked asynchronously via Starlette's run_in_threadpool() to prevent local LLM inference (15–30s) from blocking FastAPI's async event loop.

The Standardized 15-Key JSON Schema Contract

```
{ "current_title": "string", "previous_titles": ["string", ...], "current_company": "string",
"previous_companies": ["string", ...], "years_experience": float, "seniority": "junior | mid | senior | lead
| principal", "primary_domain": "string", "industries": ["string", ...], "core_skills": ["string", ...],
"secondary_skills": ["string", ...], "tools": ["string", ...], "leadership_experience": boolean,
"key_achievements": ["string", ...], "location": "string | null", "summary": "string (max 300 chars)" }
```

Null Handling: If a field is missing from the resume, the model outputs null or []. It never invents or guesses facts.

4. Stages 4 & 5: Relational State & Dense Vector Embedding

Source Files: backend/app/models.py, backend/app/services/embeddings.py

- **PostgreSQL ACID Persistence:** The extracted JSON is validated against Pydantic v2 (ParsedResumeData). On successful validation, it is stored in the candidates table under parsed_data (JSONB) with parsed_status = 'parsed'. Multi-tenant company IDs prevent cross-organization data leakage.
- **Deterministic Representation String:** To prevent semantic dilution from resume filler words, the embedding engine constructs a locked, structured representation string directly from the 15 keys:

```
Semantic Locked Representation String:
Title: {current_title} | Seniority: {seniority} | Domain: {primary_domain}
Skills: {core_skills} | Tools: {tools} | Experience: {years_experience} yrs
Achievements: {key_achievements}
```

- **Embedding Model (BAAI/bge-large-en-v1.5):** 1,024 dense semantic vector dimensions, 512 maximum token sequence length. Pre-trained on hundreds of millions of semantic text pairs using InfoNCE contrastive learning.
- **L2 Vector Normalization:** All vectors are normalized to unit Euclidean sphere length (**||v||■ = 1.0**). In unit space, dot product computation is mathematically identical to Cosine Similarity.

5. Stages 6 & 7: ChromaDB Indexing & Semantic Search Retrieval

Source Files: backend/app/services/embeddings.py, backend/app/routers/jobs.py

- **ChromaDB Vector Store:** ReSumo initializes a local persistent Chroma client storing vectors in the candidates collection. ChromaDB maintains a **Hierarchical Navigable Small World (HNSW)** graph index, providing logarithmic *O(log N)* nearest-neighbor retrieval.
- **Dual Database Architecture Rationale:** PostgreSQL serves as the ACID transactional source of truth (users, applications, foreign keys, audit logs), while ChromaDB acts as a dedicated ANN vector engine. Decoupling vector search from relational tables prevents database connection pool starvation.
- **Job Requisition Embedding:** When HR posts a job opening, its title, required seniority, and description are embedded into a 1,024-dimensional query vector **q**.
- **Cosine Similarity Ranking:** ChromaDB queries all candidate vectors **d**ᵢ in Cosine Space:

```
Cosine Similarity Metric:
    Cosine Sim(q, dᵢ) = (q · dᵢ) / ( ||q||₂ · ||dᵢ||₂ )
In ChromaDB: Distance = 1.0 - Cosine Similarity. The candidate match percentage is computed as: Match % = (1.0 - Distance) × 100%.
```

- **Threshold & Fallback Filtering:** Returns candidates matching Cosine Sim >= MIN_SIMILARITY (e.g. 0.50) up to TOP_K. **Chroma Fallback Path:** If Stage 3 extraction failed due to a damaged resume, ReSumo embeds the candidate's raw plaintext into ChromaDB, ensuring the candidate remains retrievable in search results.

6. Stages 8 & 9: Explainable AI (XAI) & Tiered HITL Governance

Source Files: backend/app/routers/jobs.py, backend/app/routers/approvals.py, frontend/src/features/recruiter/RecruiterJobsPage.tsx

- **Explainable AI (XAI) Match Rationale:** Unlike black-box screening systems that output an ungrounded score, ReSumo's RAG pipeline passes the top-matched candidate profile and job requisition into an LLM with strict grounding directives. It outputs a clear justification citing exact technical overlap, seniority alignment, and specific skill gaps.
- **Tiered Human-in-the-Loop (HITL) Policy:** ReSumo guarantees that AI never makes hiring decisions autonomously. The AI acts purely as a decision-support copilot. Shortlisting triggers a deterministic multi-tier approval state machine:

Candidate Seniority	Designated Human Approver	Verification Policy & Approval Action
Junior / Mid	Hiring Manager (HM)	Validates core skills overlap, project portfolio, and coding test.
Senior / Lead	Engineering Director / VP	Reviews compensation bounds, architectural scope, and leadership.
Principal / Executive	Chief Executive Officer (CEO)	Final sign-off on executive compensation, equity, and strategic alignment.

7. Stage 10: AI Interview Console & 5-Pillar Onboarding

- **Dynamic Interview Console:** Generates customized technical interview questions tailored to the candidate's specific parsed resume gaps, paired with a 5-dimension rubric scorecard and dual-interviewer consensus calculation.
- **Automated Onboarding Tracker:** Upon candidate offer acceptance, ReSumo automatically provisions and tracks 5 operational onboarding pillars: (1) IT & Hardware Access, (2) Legal & Compliance, (3) Banking & Payroll, (4) Corporate Orientation, and (5) HRMS Profile Integration.

8. Faculty Viva Defense Cheatsheet (Quick Reference)

Q1: Why Full SFT instead of LoRA or QLoRA?
A: LoRA and QLoRA freeze base weights and only train small low-rank adapter matrices (r=16), limiting the model's structural adaptation capacity. With Full SFT, we updated 100% of all transformer parameters across all layers. This enabled the model to deeply assimilate complex recruitment terminology, varied resume structures, and strict 15-key JSON syntax without adapter bottlenecks or inference overhead. Once trained, the full weights were converted to 8-bit GGUF for rapid local execution.

Q2: Why use both ChromaDB and PostgreSQL? Why not pgvector alone?
A: PostgreSQL is our ACID-compliant transactional source of truth for users, jobs, multi-tier approvals, and immutable audit logs. ChromaDB is our dedicated, high-performance vector database optimized specifically for HNSW Approximate Nearest Neighbor (ANN) search. Decoupling transactional queries from high-dimensional vector math prevents connection pool starvation and ensures high concurrency.

Q3: How does the system prevent LLM hallucinations?
A: In Stage 3, the fine-tuned parser is trained strictly on target JSON tokens with reasoning traces disabled, and falls back to null/empty lists for missing data. In Stage 8, RAG prompts instruct the LLM to ground its match rationales exclusively in the retrieved candidate profile and job specification.